Loading, Error & Empty States

8.2 – Loading, Error & Empty States

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"How do you handle loading and error states?"	🟢🟡	★★★★★
"What is a skeleton screen?"	🟡	★★★☆☆
"How do you handle empty states?"	🟡	★★★☆☆

The Four States of a Data Component

Every component that fetches data cycles through these states. Missing any one of them creates a broken user experience.

JSX Copy

```
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    setError(null);

    fetch('/api/users')
      .then(res => {
        if (!res.ok) throw new Error(`Server responded with ${res.status}`
        return res.json();
      })
      .then(data => { if (!cancelled) { setUsers(data); setLoading(false);
      .catch(err => { if (!cancelled) { setError(err.message); setLoading(

    return () => { cancelled = true; };
  }, []);

  // The cascade order matters: loading, then error, then empty, then data
  if (loading) return <UserListSkeleton />;
  if (error) return <ErrorCard message={error} onRetry={() => window.locat
  if (!users || users.length === 0) return <EmptyState message="No users f

  return (
    <ul>
      {users.map(user => <li key={user.id}>{user.name}</li>)}
    </ul>
  );
}
```

Why this order?

Check **loading first** so you never show stale data during a refetch. Check **error next** because an error with stale data showing is confusing. Check **empty** before data because an empty array is technically "successful" but needs a different UI.

Skeleton screens vs spinners

Skeleton screens show the shape of the content that's loading. The user's brain starts processing the layout immediately, which makes the load feel 30-40% faster (perception, not actual speed). Every modern app (YouTube, Facebook, LinkedIn) uses skeletons.

The Reusable Pattern

Extract the loading/error/data logic into a custom hook (covered in 2.4) or use React Query (8.1). The component should only care about rendering.

JSX Copy

```jsx
// With React Query: zero boilerplate for loading/error
function UserList() {
  const { data: users, isLoading, error } = useQuery({
    queryKey: ['users'],
    queryFn: () => fetch('/api/users').then(r => r.json()),
  });

  if (isLoading) return <UserListSkeleton />;
  if (error) return <ErrorCard message={error.message} />;
  if (users.length === 0) return <EmptyState message="No users yet." />;

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

✅ Key Takeaways

- Every data component needs **four states**: loading, error, empty, success.
- Check in order: **loading, error, empty, then data.**
- Use **skeleton screens** over spinners for perceived speed.

blank.

QUICK CHECK — no marks, just for you.

1. The lesson recommends skeleton screens over spinners. What is the stated reason?

- ○ Skeletons remove the need to track a separate loading flag, since the component renders the final layout either way
- ○ Skeletons avoid the layout shift that spinners cause when data finally arrives, which is a hard requirement of the four-state cascade
- ○ Skeletons show the shape of the incoming content, so the load feels faster even though actual speed is unchanged
- ○ Skeletons let the browser start painting real data before the request resolves, cutting actual load time

Check answer

2. In the lesson's cascade, the loading check comes before the error check and the data render. What problem does putting loading first specifically prevent?

- ○ Firing the effect's fetch a second time before the first one settles
- ○ Rendering the empty state for a response that has not been parsed as JSON yet
- ○ Showing stale data from a previous fetch while a refetch is in flight
- ○ Leaking a state update after the component has already unmounted

Check answer

3. [illegible] olves successfully with the array []. Per the lesson's pattern, what should UserList render, and why?

- ○ The skeleton, because the component cannot distinguish an empty array from data that has not arrived
- ○ The empty state, because an empty array is a successful response but needs its own UI
- ○ The error state, because a response with no records means the query failed to match anything
- ○ An empty <ul>, because the success branch already handles zero items by mapping nothing

Check answer

‹ Previous
Fetching Data: fetch vs axios vs React Query

Next ›
Authentication & JWT in React

LEARN MORE

ARTICLES & DOCS

- Suspense — API reference
  react.dev
- TanStack Query — request states
  github.com

VIDEOS



Loading, error, empty & success states
YouTube



Rethinking isLoading in data fetching

YouTube

Hand-picked references for this lesson.